TokenPropulsion

Task-Contract Prompt Compression and Evidence-Bounded Evaluation



Ciprian Ștefan Plești

Version 2.0.0a1 | 18 October 2026 | MIT

Independent engineering whitepaper. Not peer reviewed.

Research alpha edition. Original 10,000-word v1 paper preserved. No live provider or loaded-model study executed.

Local engineering evidence does not establish universal semantic equivalence or live xAI compatibility. This project is independent of xAI, SpaceX, and Elon Musk.

Contents

Abstract 3

1. Research question and scope 3

2. Related work and design choice 3

3. Threat model and correctness vocabulary 4

4. Conservative and adaptive paths 4

5. Structural and lexical strategies 4

6. Explicit structured documents 5

7. Optional semantic proposals 5

8. Reversible transport and decoder overhead 5

9. Counting and cost provenance 6

10. Laboratory design 6

11. Results and interpretation 6

12. Timing methodology and limitations 7

13. Provider experiments and quality gates 7

14. Privacy, resource bounds and deployment 7

15. Validation, reproducibility and release status 8

16. Future work and conclusion 8

References 9

Abstract

Prompt compression is useful only when its measured savings and its correctness assumptions are visible. TokenPropulsion 2.0 extends an existing dependency-free formatting library with task-validated transformations, explicit structured-document contracts, optional local embedding proposals, and a bounded experimental interface to the official xAI SDK. The implementation retains conservative defaults and returns the original input when a candidate fails its applicable checks. This paper reports an engineering study, not a claim of universal semantic equivalence. A project-authored synthetic dataset contains thirty cases across fifteen categories. With the unrelated local cl100k_base tokenizer, aggregate input length decreased from 8,871 to 6,249 tokens, a reduction of 29.557 percent. All declared deterministic task contracts passed; no model answer quality was measured. Ten repeats per case produced mean local compression overhead of 2.450 milliseconds, median 0.161 milliseconds, and p95 23.379 milliseconds on the recorded Windows environment. No paid provider call, real-world corpus study, loaded embedding-model evaluation, or invoice comparison was executed. The release is an alpha intended for independent evaluation.

1. Research question and scope

The principal question is whether a small auditable library can remove explicitly redundant material without silently broadening the meaning of its guarantees. Formatting preservation, task correctness, semantic equivalence, and provider billing are different observables. A string transformation can preserve the first while violating the others. The implementation therefore associates each operation with a limited contract and exposes token-count provenance. The 20-40 percent research target applies to suitable redundant workloads; it is neither an acceptance threshold for arbitrary prompts nor a promise about financial savings.

Version 1.0.0rc1 provided conservative formatting profiles, protected-input preservation, immutable results, and an optional xAI adapter. Its original paper contains exactly 10,000 whitespace-delimited words in the documented counted body. That paper, its source, historical reports, release tag, and release assets remain preserved. This document is a separate shorter research edition for the alpha implementation. It supersedes no historical measurement and does not reinterpret the earlier heuristic benchmark as exact token evidence.

2. Related work and design choice

LLMLingua investigates learned prompt compression under budget constraints [1]. LLMLingua-2 investigates a distilled token classification approach [2]. These studies motivate separating compression ratio from downstream task performance and evaluating different domains. TokenPropulsion does not execute their models, reuse their code, or claim comparable accuracy or speed. Microsoft's reference implementation carries an MIT license [3]; model weights and dependencies require separate review. A direct empirical comparison remains future work.

The present design deliberately starts with inspectable contracts rather than a learned pruning system. SentenceTransformers provides a possible proposal mechanism through an isolated extra [4]. Its Apache-2.0 library license does not automatically determine the license of every model checkpoint [5]. Local embedding similarity can suggest redundancy, but similarity does not establish whether a detail can safely be removed. All embedding candidates therefore require an independent caller-provided task validator before acceptance.

3. Threat model and correctness vocabulary

An input can contain code, exact output requirements, numbers, mathematical expressions, sensitive markers, hostile instructions, or unfamiliar scripts. The core accepts bounded Unicode text and applies conservative whole-prompt protection when recognized patterns match. Financial, medical, legal and security-sensitive markers expand the prior protections. Pattern recognition is incomplete: a protected-content test suite is evidence about defined examples, not a complete detector of sensitive meaning.

A transformation is mechanically valid when its specified string invariants hold. It is task-validated when an explicit evaluator accepts the relationship between original and candidate. It is semantically equivalent only under a much stronger interpretation that this project does not claim to decide. The confidence field is descriptive metadata, not a calibrated probability or a semantic proof. A caller can defeat task correctness by supplying a validator that accepts everything; such behavior is outside the validated contracts.

4. Conservative and adaptive paths

The original low, medium and high profiles remain available. They trim trailing ASCII spaces, normalize excessive blank lines, or collapse repeated prose spacing under the existing protection policy. Ordered nonwhitespace checks, protected identity, and nonincreasing token counts govern acceptance. Default calls retain the old signature and do not load optional packages or contact providers. The result retains original text so restore returns it exactly.

Adaptive selection is deterministic. Without an explicit validator it uses conservative formatting. With a validator and repeated paragraphs it can propose structural deduplication. It does not automatically invoke a remote model, load embeddings, or grant permission for content transmission. Advanced strategy errors are sanitized and return the original text rather than exposing exception details that might contain prompt data. Invalid token counts are rejected rather than interpreted as savings.

5. Structural and lexical strategies

Structural compression removes exact repeated paragraphs while preserving the order of retained paragraphs. This operation is valid only where the caller has declared an appropriate set-like task contract. Repetition can convey emphasis, voting weight, chronology, or instructions; duplicate bytes alone do not justify deletion. The explicit paragraph-set validator used in tests demonstrates one narrow contract and must not be generalized to arbitrary natural language.

Lexical compression contains four inspectable phrase substitutions, including replacing 'in order to' with 'to'. It also requires a task validator and passes through protection and count gates. Its purpose is to provide a bounded experimental interface rather than a general paraphraser. The synthetic keyword tasks test retention of declared task markers after substitution; they do not evaluate complex reasoning, fluency, pragmatic meaning, or downstream model behavior. The dataset category named complex_reasoning is therefore a lexical toy and must not be interpreted as a reasoning benchmark.

6. Explicit structured documents

PromptDocument separates instructions, ordered untrusted context sections, and question. Each context includes a source identifier, text, trust label and an idempotence declaration. Rendering uses JSON encoding, preventing delimiter-looking content from escaping its string representation. Compression may remove an exact earlier context only when its complete section record matches, its text is unprotected, and idempotence is explicitly true. Different sources, trust boundaries, or flags remain separate.

An independent validator checks that instructions and question are identical, retained contexts form an ordered subsequence, and each removed section is an allowed duplicate of an earlier section. This is stronger than a producer declaring its own output correct, but it still depends on the caller's idempotence assertion. Repeated context may affect model behavior even when a retrieval application intends set semantics. The contract does not establish comprehensive prompt-injection resistance or universal RAG answer preservation.

7. Optional semantic proposals

SentenceTransformerBackend requires an existing local model directory. Loading uses local-files-only behavior, disables trusted remote code, and requests safetensors. The proposal path bounds sentence count and sentence length, computes normalized embedding similarity, and suggests removal of sufficiently similar sentences. An external task validator remains mandatory. The optional dependency is not included in the default runtime and no checkpoint is bundled.

Release tests substitute a deterministic model stub to exercise adapter behavior, loading arguments and limits. No real checkpoint was loaded, no model quality was measured, and no semantic throughput claim is supported. Operators must review checkpoint provenance, configuration, license, memory requirements and hardware costs. Dependency and checkpoint safety cannot be inferred solely from the absence of remote code execution in the adapter.

8. Reversible transport and decoder overhead

A separate reversible module serializes zlib-compressed UTF-8 data as a base64 JSON envelope. Decoding enforces the one-megabyte output bound and rejects malformed, truncated, trailing and oversized streams. Exact round trips are tested. This is transport compression, not an assumption that a language model can interpret compressed bytes. A receiving program must execute the declared decoder before submitting ordinary prompt text to a model.

Wire accounting counts both the envelope and the entire decoder instruction supplied by the caller. Reporting only the compressed payload would omit material overhead. The transport path exposes decoder-required metadata and makes no provider-token savings claim. Restoration of an ordinary CompressionResult differs from this transport feature: the former returns the stored original string without asking a model to reconstruct deleted information.

9. Counting and cost provenance

The default counter is a UTF-8 byte heuristic. Tiktoken counts are exact for the selected loaded vocabulary, but cl100k_base is classified as unrelated to the chosen Grok model. Initial vocabulary acquisition requires explicit network opt-in. A compatible-local provenance category is recognized for caller-supplied versioned evidence; the release supplies no verified-compatible Grok tokenizer or compatibility certificate. Custom counters without declared provenance remain custom-unspecified.

The official xAI SDK can supply provider-exact text tokenization and provider-reported request usage [6,7]. These are distinct from local text-only counts because generation requests can include message framing and other overhead. Cached input usage and reported cost are exposed when the SDK supplies them. Actual billed cost requires invoice or account reconciliation, which is absent here. No fixed model price, model name, or claim of financial savings is embedded in the measured local report.

10. Laboratory design

The evaluation corpus is project-authored synthetic material under MIT. A manifest records its SHA-256, provenance and license. It contains thirty cases in fifteen categories, two cases per category. Eight long context cases use explicitly idempotent duplicate sections; two lexical cases use keyword contracts; twenty protected cases cover code, JSON generation, mathematics, Romanian, Arabic, mixed-language content, financial content, security-sensitive material, exact formatting and adversarial input. No confidential or real-world prompts are represented.

The laboratory compares unchanged input, current conservative formatting, the selected opt-in strategy, and an intentionally unsafe whitespace baseline. The unsafe baseline is an analytical reference and is never automatically sent to a provider. Reports include per-case counts, task checks, fallback flags, timing summaries, CSV comparisons, failure notes and an SVG chart derived directly from measured counts. The output directory must be empty so historical evidence cannot be overwritten accidentally.

11. Results and interpretation

The local exact-vocabulary study measured 8,871 original and 6,249 optimized tokens. The aggregate token-weighted reduction was 29.556983 percent. Repeated-context rows reduced by approximately 28.57-32.48 percent; lexical rows reduced by approximately 8.70-11.76 percent; twenty protected rows reduced by zero. Longer repeated documents contribute most of the aggregate count. Category-balanced averages and real traffic mixes may produce substantially different results. A high aggregate saving does not demonstrate a general-purpose semantic compressor.

Every declared deterministic task contract passed, invalid transformations were zero, and fallback was zero on this dataset. These observations mean that the implemented synthetic checks accepted those transformations. They do not mean model answers remained correct, that all important information was preserved for every conceivable task, or that Grok usage fell by the same percentage. Model output quality, actual provider-token reduction and billed cost fields remain null rather than being inferred from the local count result.

12. Timing methodology and limitations

Each case was compressed ten times using a monotonic performance counter. Mean local duration was 2.450484 milliseconds; median and p50 were 0.160750 milliseconds; p95 was 23.378860 milliseconds. Sample variance was 0.0000616111 seconds squared. A normal-approximation 95 percent interval for the mean was 1.562255-3.338713 milliseconds. This interval is descriptive: repeated observations share a machine, software state and workload, so independence and normality are not established.

The distribution is heterogeneous because long structured documents and short protected prompts have different processing costs. The pooled mean should not replace per-row latency inspection. Measurements include local proposal, validation and counting overhead as implemented in the laboratory, not provider generation latency. No streaming time-to-first-token, network round trip, endpoint concurrency, memory profile or hardware-normalized throughput experiment was executed. The environment and timestamp are recorded in the raw JSON rather than assumed portable.

13. Provider experiments and quality gates

The xAI study interface requires explicit billing authorization, a model, a RequestBudget and reviewed network permission. Before any provider request, it validates expected outputs, evaluator names, task contracts and optimized prompts for the entire dataset. It checks worst-case retry reservation requirements against remaining request and spending reservations. Original and optimized pair order alternates across repeats to reduce a simple ordering bias, but the design is not a randomized controlled trial.

Acceptance for follow-up requires observations, every optimized answer passing its declared evaluator, and no failed pairs. Thus two equally wrong variants are rejected, and an improvement cannot offset a separate regression. Exact-string and strict canonical-JSON evaluators are implemented; they are not universal quality metrics. A separately authorized LLM judge offers a nondeterministic opinion only and cannot approve compression. Judges can be prompt-injected or wrong. No paid study was executed for this release.

14. Privacy, resource bounds and deployment

Core compression makes no network requests. The REST factory requires an ASCII bearer token of at least thirty-two characters, performs constant-time comparison, rejects encoded bodies and extra request fields, bounds input bytes, limits request frequency per process, and applies a ten-second body-read timeout. Error messages suppress input details. These controls do not supply TLS, a shared distributed limiter, durable authorization policy, or complete denial-of-service protection. Production deployment needs an appropriately configured reverse proxy, worker concurrency limits and operational review.

Result objects intentionally contain original and optimized prompts for local auditability. Applications must decide whether they may persist them. Experiment reports omit prompt and response text by design, but external SDK, proxy or application logging can still expose content. Local reservation budgets are not provider-side hard caps: a single request may cost more than its reservation, and failed or retried requests may still be charged. Configure provider account controls and review all content before remote transmission.

15. Validation, reproducibility and release status

The repository retains the original regression suite and adds tests for advanced strategies, document boundaries, reversible decoder attacks, token provenance, budget limits, mocked provider usage and costs, task quality acceptance and REST limits. Final execution evidence resides under AUDIT/evidence-v2 rather than replacing historical v1 reports. Ruff, mypy, coverage, Bandit, dependency auditing, package builds and Twine form release gates. The eight audit documents distinguish implementation evidence, experimental features and external prerequisites. The audit is an internal second-pass engineering review, not an independent certification.

Reproducibility means providing source, dataset hashes, explicit counting provenance, commands and checksums. It does not promise identical timing on another machine or byte-identical PDF rebuilds. The dependency-free wheel must work in a fresh environment without optional imports. GitHub prerelease publication is separate from stable or PyPI publication. The alpha remains unsuitable for a production-readiness claim until representative tasks, live provider behavior and optional loaded-model behavior receive independent evaluation.

16. Future work and conclusion

Priority work includes a licensed real-world dataset, stronger task-specific evaluators, controlled live provider trials, per-domain quality thresholds, loaded embedding-model assessment, and direct comparisons with learned compression baselines. Streaming TTFT, invoice reconciliation, memory measurements and distributed API abuse controls remain unimplemented. A compatible local tokenizer should be accepted only with versioned provider evidence, not because it correlates with an unrelated vocabulary on a few strings.

The demonstrated contribution is a bounded implementation whose successful synthetic reductions can be traced to explicit contracts. The 29.557 percent local-vocabulary result supports evaluating redundant set-like contexts further. It does not establish general semantic preservation, financial savings or xAI endorsement. By preserving historical evidence, exposing fallback and provenance, and marking unknown outcomes as unknown, the release provides a concrete basis for independent engineering review.

References

[1] Jiang et al. (2023). LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. EMNLP. https://aclanthology.org/2023.emnlp-main.825/

[2] Pan et al. (2024). LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. Findings of ACL. https://aclanthology.org/2024.findings-acl.57/

[3] Microsoft. LLMLingua implementation license, MIT. https://github.com/microsoft/LLMLingua/blob/main/LICENSE

[4] SentenceTransformers. SentenceTransformer API reference. https://www.sbert.net/docs/package_reference/sentence_transformer/model.html

[5] Hugging Face. SentenceTransformers license, Apache-2.0. https://github.com/huggingface/sentence-transformers/blob/main/LICENSE

[6] xAI. Official Python SDK. https://github.com/xai-org/xai-sdk-python

[7] xAI. Generate text documentation. https://docs.x.ai/developers/model-capabilities/text/generate-text

[8] TokenPropulsion. Measured synthetic laboratory, version 2.0.0a1. Repository path: benchmarks/results/v2.0.0a1/optimized.json. Dataset SHA-256: 7418f1d284438b60cd0a81eba84731e4a2c833ea803c320a0bb8c89ed5a96b83.